

The Alcesat sub-granule index

design, on-disk layout, and the contents of every entry

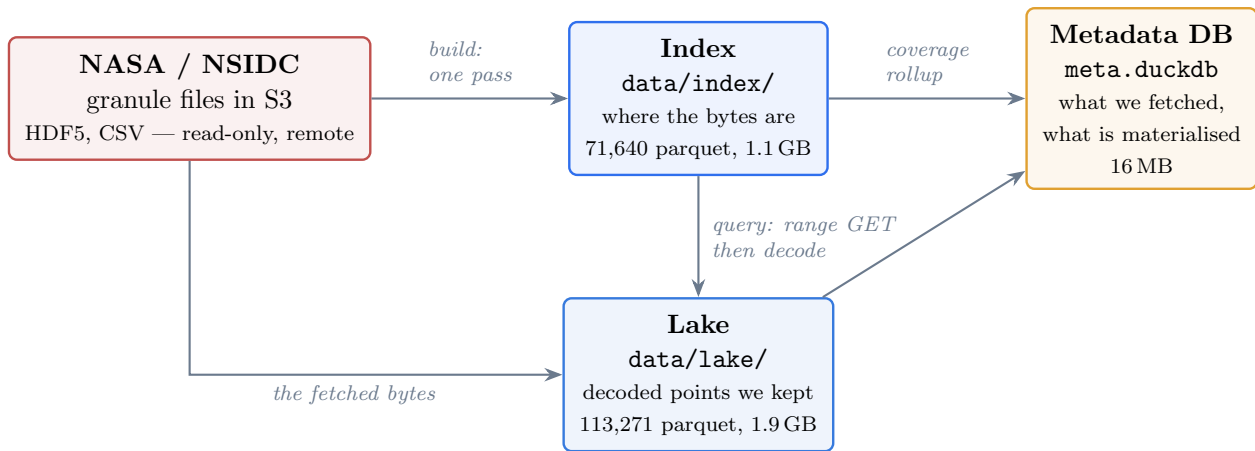
2026-09-09

Abstract

Alcesat answers questions of the form “give me every altimetry point from four missions inside this box” without downloading whole granules. It does that with a **sub-granule index**: a small table that records, for each spatial cell of each granule, the exact *byte ranges* inside the remote file that hold the points for that cell. A query becomes a set of HTTP range GETs rather than a set of file downloads. This document describes the three stores that make that work — the index, the lake, and the metadata database — down to the directory tree, the file naming, and the meaning of every column. Section 7 covers a design question that keeps coming up: whether H3’s cell hierarchy should carry the rollup metadata. All figures for size and time are measured on the deployed `us-west-2` instance, 2026-09-09.

1 The three stores

The system keeps three things on disk, with sharply different jobs. Confusing them is the source of most of the bugs this design has produced, so it is worth being blunt about the split.



The index is an *address book*. It never holds science values — only the coordinates of bytes. It is built once per granule and is then immutable. Its unit is (granule, beam, chunk, H3 cell).

The lake is a *cache of decoded points*, in the projected form the application wants. Losing it costs time, never truth: anything missing is re-fetched. Its unit is one Parquet file per (granule, beam, chunk) *inside* one H3 cell directory.

The metadata DB records *what has been fetched* (so a chunk is never fetched twice) and, since 2026-09-09, *what is materialised* (so sizing the lake does not require walking it). These are two different questions and the distinction matters — see §5.

2 H3 as the addressing key

Every store is keyed on an H3 cell index — a 64-bit integer naming a hexagon on a global grid. Resolutions in use:

Collection	Product	Index res	Lake res
ATL06	ICESat-2 ATL06 land ice	5	5
ATL03	ICESat-2 ATL03 photons	6	6
GLAS	ICESat/GLAS GLAH06	5	5
ICESSN	IceBridge ATM ILATM2	5	5

The dense photon product (ATL03) is indexed one resolution finer because a res-5 cell holds far too many photons to be a useful fetch unit. Index and lake resolutions are deliberately *equal* within a collection: a lake file is written at the same resolution its index row named, so a later query for a different sub-box within the same cell re-filters correctly instead of returning a partial cell.

One subtlety that is easy to get wrong. H3’s hierarchy is *not* exactly nested. A resolution-5 hexagon is not the union of its resolution-6 children; the aperture-7 subdivision makes children straddle the parent’s boundary. What *is* exact is the *index* relation: `cellToParent(c,r)` is a total function on cell IDs, so every cell has exactly one ancestor at each coarser resolution. Any rollup defined over *cell IDs* is therefore well defined; only a rollup interpreted as “the metadata of the parent’s geographic area” inherits the boundary error. Section 7 depends on this distinction.

3 The sub-granule index

3.1 Directory layout

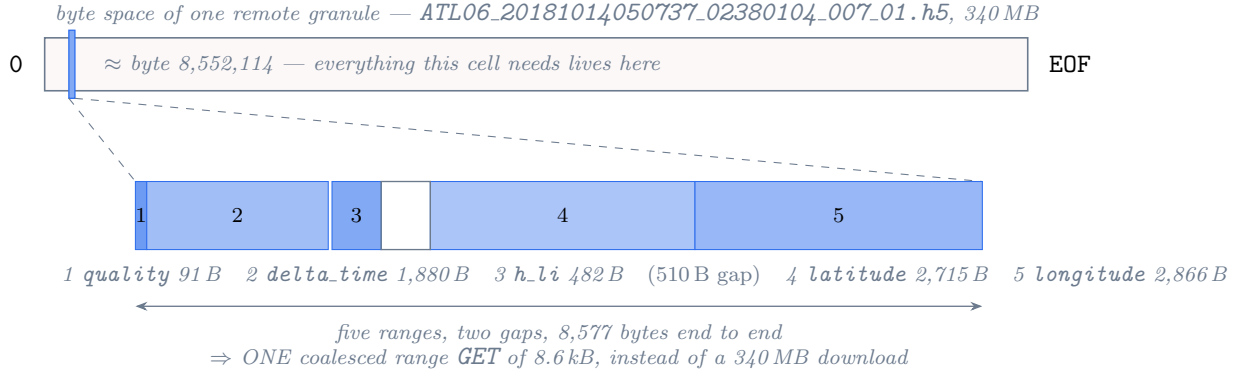
```
data/index/
├── atl06/res5/
│   ├── ATL06_20181014050737_02380104_007_01.h5.parquet  one per granule (33,064)
│   ├── ...
│   ├── _coverage/manifest.parquet  rolled-up (cell, granule, ym)
│   ├── _coverage/meta.json  freshness stamp
│   └── _build.json  the cell list this build targets
├── glas/res5/  4,741 granule parquet + _coverage/
├── icesn/res5/  33,793 granule parquet + _coverage/
├── atl03/  42 granule parquet + _coverage/ (no res subdir)
└── meta.duckdb  16 MB
```

The per-collection directory is *flat*: one Parquet file per granule, named exactly `<granule>.parquet` (the granule name keeps its `.h5` / `.csv` suffix, so the stem is `..._007_01.h5`). Two consequences follow, and both have bitten:

1. Files are written **tmp-then-rename**, so a reader never sees a torn Parquet, and the *directory* mtime moves whenever a granule lands. That single `stat()` is the freshness gate used everywhere downstream (§3.3).
2. Anything that answers a question by opening every file in the directory does work proportional to the *store*, not the *request*. At 33,064 files that is 43.5 s. The `_coverage/` rollup exists precisely to stop that, and lives in a *subdirectory* so that a glob of `*.parquet` (which the fetch path uses) cannot pick up a schema-mismatched file — and so that writing it does not invalidate its own freshness stamp.

3.2 What a row contains: byte-range addressing

This is the heart of the design. A granule is an HDF5 file of a few hundred MB whose datasets are stored as compressed chunks. For each contiguous run of measurements that falls in one H3 cell, the index records where the *compressed bytes* of every dataset it will need begin and end, plus everything required to decode them without opening the file.



the index row that says so

```
h3_cell=598985434483130367 beam=gt1l chunk_index=0
seg_start=0 seg_end=410 lat_min=80.860 lat_max=81.257
latitude_offset=8555110 latitude_size=2715
latitude_filters=gzip latitude_dtype=float64 latitude_ncols=1
... and the same sextet for longitude, h_li, delta_time, atl06_quality_summary
```

offset/size give the range; filters says how to inflate it (gzip, or shuffle, gzip for the int8 columns); dtype/ncols say how to view the result. Nothing here requires opening the file. ATL03's ranges are not adjacent — 3.28 GB apart in the example granule — which is why coalescing is bounded by a gap threshold rather than unconditional.

The addressing comes in three flavours, one per source format:

Collection	Row grain	Addressing columns
ATL06, ATL03	(granule, beam, chunk, cell)	per dataset: <code>_offset _size _filters _dtype _ncols _mask</code> . HDF5 chunk ranges. <code>_ncols</code> matters: ATL03's <code>signal_conf_ph</code> is $n \times 5$.
GLAS	(granule, beam=na, chunk, cell)	per dataset: <code>_offset _size _filters _dtype _mask _fill</code> . Carries an explicit <code>_fill</code> because GLAH06 encodes no-data as 1.798×10^{308} , not NaN.
ICESSN	(granule, cell)	<code>byte_start byte_end n_lines</code> . Plain CSV — a line range, no decompression.

Every collection additionally carries the columns in Table 1.

Table 1: Columns common to all index rows.

Column	Type	Meaning
granule	string	source file name, suffix included; the join key everywhere
url	string	HTTPS location (used out of region, via presigned redirect)
s3url	string	s3:// location (used in us-west-2: no presign, free egress)
revision	string	NSIDC revision id, so a re-issued granule is distinguishable
h3_cell	uint64	the cell this row addresses, at the collection's index resolution
lat_min/max	double	bounding box of <i>this row's</i> points, for pre-filtering before decode
lon_min/max	double	
chunk_index	int64	which decode unit this is within the granule (absent for ICESSN)

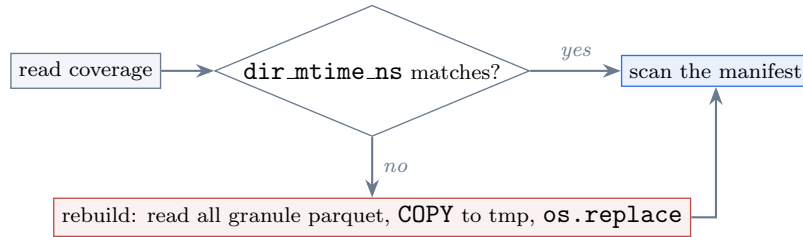
Mission-specific extras: ATL06/ATL03 carry `sc_orient`, `sdp_epoch`, `beam`, `strong`, `cycle`, `rgt` and a segment or photon range (`seg_start/seg_end`, `ph_start/ph_end`); GLAS and ICESSN carry `gdate` (YYYYMMDD) because they have no repeat-cycle number. A typical granule yields a small number of rows — 18 for the ATL06 example above, 7 for ATL03, 25 for GLAS, 2 for ICESSN — so the whole index is 1.1 GB for 71,640 granules.

3.3 The coverage manifest

Answering “which cells does this collection cover, and over what period?” from the per-granule files means opening all of them. The manifest is the rollup that avoids it:

<index dir>/_coverage/manifest.parquet — DISTINCT triples		
<code>h3_cell</code>	<code>uint64</code>	cell at the index resolution
<code>granule</code>	<code>string</code>	source granule
<code>ym</code>	<code>string</code>	YYYY-MM, derived from <code>gdate</code> (GLAS/ICESN) or parsed out of the granule <i>name</i> (ATL0x)

For ATL06 that is 2,261,439 rows in 27 MB, against 33,064 files and 1.1 GB. The accompanying `meta.json` holds `{n_source_files, dir_mtime_ns, built_at}`, and the freshness protocol is:



The stamp is written before the sources are read, so a granule landing mid-build leaves the recorded mtime older than the directory's and the next read rebuilds. Stale-and-corrected, never silently-missing. Writes land inside `_coverage/`, which does not touch the parent's mtime, so the manifest cannot invalidate itself.

Known lag. Because the rebuild is lazy, the manifest can trail the directory by a few granules between builds — measured 2026-09-09: 33,053 vs 33,064 (ATL06), 4,649 vs 4,741 (GLAS), 33,578 vs 33,793 (ICESN). The missing granules fall in already-covered cells, so the *cell set* is unaffected (14,482 / 23,142 / 7,782 — identical either way). Counts that must be exact, notably the “index build *N*% done” readout, therefore come from a directory listing rather than from the manifest.

4 The lake

4.1 Layout and naming

```

data/lake/mission=ATL06/h3_cell=599100951923523583/
  ATL06_20181119053215_07880105_007_01__gt1l__c5.parquet  <- granule __ beam __ chunk
  ATL06_20181119053215_07880105_007_01__gt1r__c5.parquet
  ...
  (656 files in this cell)
data/lake/mission=ICESAT2/h3_cell=604538106045005823/
  ATL03_20210830193051_10411206_007_01.h5__gt2r.parquet  <- granule __ beam (NO chunk)
  
```

Hive-style partitioning, so DuckDB prunes on `mission=` and `h3_cell=` without reading anything. The file name is the write's identity, which makes re-materialisation idempotent by construction.

The ICESat-2 photon path names files differently, and it matters. `write_point_chunk` (ATL06 / GLAS / ICESN) emits one file per (granule, beam, chunk) — so one coverage row corresponds to exactly one file. `write_photons` (ATL03, stored under `mission=ICESAT2`) omits the chunk from the name, so *many* coverage rows map to *one* file: 390 rows against 363 files. Any scheme that derives per-file quantities by summing per-row quantities is wrong on this path.

4.2 What a lake row contains

Column	Type	Meaning
<code>native_lon</code> , <code>native_lat</code>	double	position as published, never rewritten
<code>native_height</code>	double	elevation in the source’s own datum
<code>t</code>	timestamp[ms]	observation time
<code>source_granule</code> , <code>beam</code>	dictionary	provenance, dictionary-encoded (one value per file)
<code>source_chunk_index</code>	int32	which fetch unit produced this row
<i>ATL06 adds</i>		
<code>quality</code>	int8	<code>atl06_quality_summary</code>
<i>ICESSN adds</i>		
<code>sn_slope</code> , <code>we_slope</code>	double	along/across-track surface slope
<i>ICESAT2 (ATL03 photons) adds</i>		
<code>row_id</code>	uint64	<code>crc32(granule/beam) << 32</code> photon index — a deterministic surrogate key
<code>height_ref</code> , <code>native_frame</code>	dictionary	e.g. <code>WGS84_ellipsoid</code> , <code>ITRF2014</code>
<code>coreg_lon</code> , <code>coreg_lat</code> , <code>coreg_epoch</code>	double	co-registered position at <code>COMMON_EPOCH = 2005.0</code> , materialised at ingest
<code>signal_conf_landice</code>	int8	photon confidence
<code>photon_index</code>	int64	index within the granule/beam

Rows are written in along-track order with `ROW_GROUP_ROWS = 65,536`, so each row group carries tight lat/lon statistics and DuckDB skips the groups a bounding-box predicate cannot touch. Compression is `zstd`.

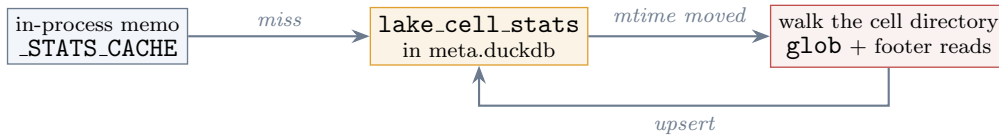
5 The metadata database

`data/index/meta.duckdb` holds two tables that answer two *different* questions. Conflating them is the single most productive source of bugs in this subsystem.

<code>coverage_cells</code> — <i>what has been fetched</i> . 118,157 rows.		
<code>mission</code>	VARCHAR	ATL06 / GLAS / ICESN / ICESAT2
<code>granule</code>	VARCHAR	
<code>beam</code>	VARCHAR	na where the source has no beams
<code>chunk_index</code>	INTEGER	
<code>h3_cell</code>	UBIGINT	
<code>ingested_at</code>	TIMESTAMP	
PK (mission, granule, beam, chunk_index, h3_cell)		
<code>lake_cell_stats</code> — <i>what is materialised</i> . One row per cell.		
<code>mission</code> , <code>h3_cell</code>	VARCHAR, UBIGINT	PK
<code>dir_mtime_ns</code>	HUGEINT	the cell directory’s mtime this rollup was computed at
<code>rows_known</code>	BOOLEAN	false when computed by a cheap pass that skipped row counts
<code>files</code> , <code>n_rows</code> , <code>n_bytes</code>	INTEGER, BIGINT, BIGINT	
<code>granules</code> , <code>beams</code>	VARCHAR[]	

Why `coverage_cells` cannot answer “how big is my lake”. It deliberately records cells a chunk *wanted* but wrote nothing to. `index_atl06` marks every wanted cell of a fetched chunk “so an all-fill cell is not re-fetched forever”; the writer then drops non-finite positions and, for ATL06, segments with `quality ≠ 0`. Measured on the deployed lake: **4,373 of 107,363 ATL06 rows (4.1%) name a file that does not exist**, spread over 184 cells and 922 granules — and *every* file on disk has a row (zero unmarked). It is not drift and not eviction; it is a faithful record of a different fact. Hence the separate rollup table.

5.1 The three-tier read of the rollup



Gate at every tier is one *stat()* of the cell directory. The database is opened only when something actually changed, so a steady-state poll touches no database at all. Cost on the box: 14.05 s → 0.27 s across a process restart, identical output.

6 Read paths and what they cost

Measured on the deployed `t3.large` in `us-west-2`, 2026-09-09, against 102,990 lake files and 71,640 index parquet.

Call	Walking files	From metadata	Source of the fast answer
<code>lake_summary(ATL06)</code>	7.09 s	0.27 s	<code>lake_cell_stats</code> + per-cell mtime
<code>index_status(ATL06)</code>	44.29 s	0.47 s	<code>_coverage/manifest.parquet</code>
<code>index_status(ATL03)</code>	0.05 s	0.01 s	
<code>index_status(GLAS)</code>	7.21 s	0.18 s	
<code>index_status(ICESSN)</code>	22.95 s	0.11 s	
one page load	81.58 s	1.04 s	

The recurring failure mode this table represents is worth naming: *work proportional to the store rather than to the request*. It has been found and fixed five times in this codebase. It is invisible on a developer machine, where the store is 100× smaller, and it does not degrade gracefully — an endpoint slower than its own polling interval does not become slow, it becomes *never*, because the polls stack.

7 Should the rollup live in the H3 hierarchy?

The tempting design is to store metadata at several resolutions — a res-3 row summarising its res-5 members — so that a zoomed-out view reads few rows and a zoomed-in view reads exact ones. As §2 noted, the *ID* hierarchy is exact even though the geometry is not, so this is well defined. The question is whether it pays.

Which aggregates survive a rollup.

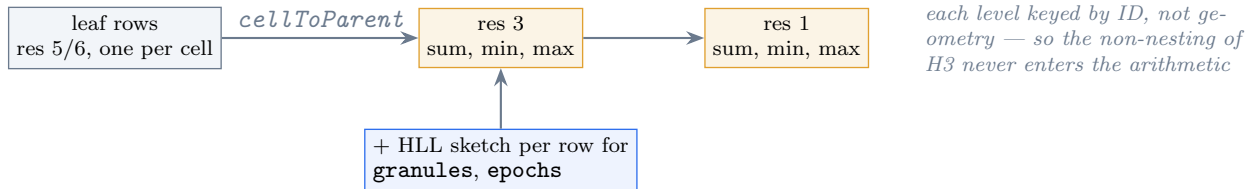
Quantity	Decomposable?	Why
<code>n_bytes</code> , <code>n_rows</code> , <code>files</code>	yes, sum	each file belongs to exactly one leaf cell
<code>ym_min</code> , <code>ym_max</code> , hence span	yes, min/max	min and max are associative
<code>granules</code> (distinct count)	no	one granule crosses many cells; summing children over-counts the shared ones
<code>epochs</code> (distinct <code>ym</code>)	no	same reason

This is not hypothetical. The client already rolls index cells up to the display grid, and it does so with `e.g += x.g` — summing a distinct-granule count. A granule crossing two res-5 cells is counted twice in their res-4 parent. The tooltip’s granule figure is therefore an over-count at zoom levels coarser than the index resolution. Span and epochs, which that same rollup takes as a `max`, are correct.

Recommendation: not yet, and not as stored rows. Three reasons.

1. **The leaf table is small enough to aggregate on demand.** The rollup is `GROUP BY cellToParent(h3_cell, r)` over 185 lake cells or 14,482 index cells — microseconds in DuckDB. Materialising ancestors optimises something that is not slow.
2. **Invalidation multiplies.** The correctness of the present design rests on one cheap, honest signal: the cell directory's mtime. A stored ancestor has no directory and therefore no such signal; every leaf write would have to invalidate $O(\log)$ ancestors explicitly, and any writer that forgot would leave a silently wrong number with nothing to self-heal it. The current tiers degrade to a slow-but-correct walk; a stale ancestor degrades to a wrong answer.
3. **The distinct-counts would need sketches, not integers.** Doing this properly means storing an HLL sketch per cell per level so `granules` and `epochs` merge correctly. That is the right answer at scale and clearly disproportionate now.

The threshold to watch. Res-5 covers the globe in roughly 2×10^6 cells. Today's index occupies 14,482 of them for ATL06 — Greenland and Antarctica at partial build. A whole-cryosphere index at res 5, or any move to res 7–8 for ATL03, puts the leaf count two to three orders of magnitude higher, and at that point the on-demand `GROUP BY` stops being free. The migration is then:



8 Invariants

The following are relied on throughout, and each has been violated at least once:

1. **Coverage is marked only after the file is on disk.** A crash loses cache, never truth — an unmarked chunk is simply re-fetched.
2. **Index files land by rename.** Readers never see a torn Parquet, and the directory mtime is a sound change signal.
3. **A lake file is written at the collection's own index resolution,** never at the planner's. A partial cell is then impossible by construction.
4. **Files are the source of truth for bytes and rows;** the metadata database is a cache of that truth, gated on a directory mtime, and never the other way round.
5. **coverage_cells is a superset of the materialised set,** by design. Counting files from it over-counts by $\approx 4\%$.